

تمرین سری پنجم

یادگیری ماشین مقدماتی



پیاده‌سازی مدل زبانی Mini-GPT با JAX

توسعه و آموزش معماری ترانسفورمر از پایه، شامل پیاده‌سازی مکانیزم توجه (Attention) و بهینه‌سازی مدل برای تولید متون ادبی.

نام دانشجو: حامد بهروزی مقدم - آیدین صحنه - مانی وفاپور

شماره دانشجویی: ۴۰۲۱۵۶۳۳ - ۴۰۱۲۰۲۴۳ - ۴۰۱۲۳۷۸۳

استاد درس: دکتر نیکوفرد

نیمسال: ۱۴۰۴-۱۴۰۵



فهرست مطالب

۲	۱	گزارش کامل فاز اول پروژه: از کاراکترها تا تنسورها
۲	۱.۱	بررسی قابلیت‌های محاسباتی فریم‌ورک JAX
۳	۲.۱	پیاده‌سازی سافت‌مکس پایدار (Numerically Stable Softmax)
۴	۳.۱	بارگذاری و تحلیل دادگان Tiny Shakespeare
۵	۴.۱	توسعه توکنایزر کاراکتری (Character-level Tokenizer)
۵	۵.۱	تولید دسته‌های داده (Data Loader / get_batch)
۷	۶.۱	جمع‌بندی
۷	۲	پیاده‌سازی مکانیزم توجه به خود علیتی (Self-Attention Causal)
۷	۱.۲	نگاشت خطی و استخراج تنسورهای پرس‌وجو، کلید و مقدار
۸	۲.۲	منطق ریاضی ماسک‌گذاری علیتی (Causal Masking Logic)
۹	۳.۲	پیاده‌سازی کامل هسته محاسباتی Self-Attention
۱۰	۴.۲	ارزیابی و تحلیل مهندسی مسافت نگاه به عقب (Look-back Distance)
۱۲	۵.۲	جمع‌بندی محاسباتی فاز دوم
۱۲	۳	فاز سوم: توجه چندسر (Multi-Head) و بلاک ترانسفورمر
۱۳	۱.۳	توسعه مکانیزم توجه چندسر (Multi-Head Attention)
۱۵	۲.۳	شبکه عصبی فیدفوروارد (MLP)
۱۶	۳.۳	مونتاژ نهایی: ساخت بلاک ترانسفورمر (Transformer Block)
۱۷	۴.۳	تحلیل جریان پس‌ماند (Residual Stream Analysis)
۱۸	۵.۳	جمع‌بندی فاز سوم
۱۹	۴	فاز چهارم: مونتاژ نهایی و آموزش مدل Mini-GPT
۱۹	۱.۴	تکمیل معماری شبکه (Mini-GPT Assembly)
۲۰	۲.۴	تعریف تابع زیان (Objective Function)
۲۲	۳.۴	توسعه موتور آموزش با JAX JIT و Optax
۲۳	۴.۴	تحلیل نتایج آموزش و همگرایی شبکه
۲۴	۵.۴	جمع‌بندی فاز چهارم
۲۴	۵	فاز نهایی: تولید متن و نمونه‌برداری هوشمند (Sampling)
۲۴	۱.۵	فیلترینگ هوشمند احتمالات (Top-P و Top-K)
۲۶	۲.۵	حلقه اصلی تولید متن (Loop Generate The)
۲۷	۳.۵	تحلیل نتایج تولید شده
۲۸	۴.۵	جمع‌بندی کل پروژه



۱ گزارش کامل فاز اول پروژه: از کاراکترها تا تنسورها

هدف فاز اول پروژه (Part 1 - The Backstory)

هدف اصلی در این بخش (نوت‌بوک init.ipynb)، ایجاد یک زیرساخت قوی و پایه برای مدل‌سازی زبانی (Language Modeling) در پلتفرم اسنپ با استفاده از فریم‌ورک JAX است. مدل‌های GPT بر مبنای معماری ترانسفورمر از نوع Decoder-only کار می‌کنند که در آن هر توکن تنها به گذشته توجه کند، نه آینده. در این فاز ما روی تبدیل متن خام به ورودی‌های عددی قابل فهم برای شبکه تمرکز کردیم. این فرآیند شامل بررسی قابلیت‌های JAX، پیاده‌سازی فرمول‌های پایدار ریاضی (Stable Softmax)، توسعه یک توکنایزر کاراکتری (Character Tokenizer) و در نهایت تولید دسته‌های داده (Batched Prediction Pairs) به منظور آموزش شبکه بود.

۱.۱ بررسی قابلیت‌های محاسباتی فریم‌ورک JAX

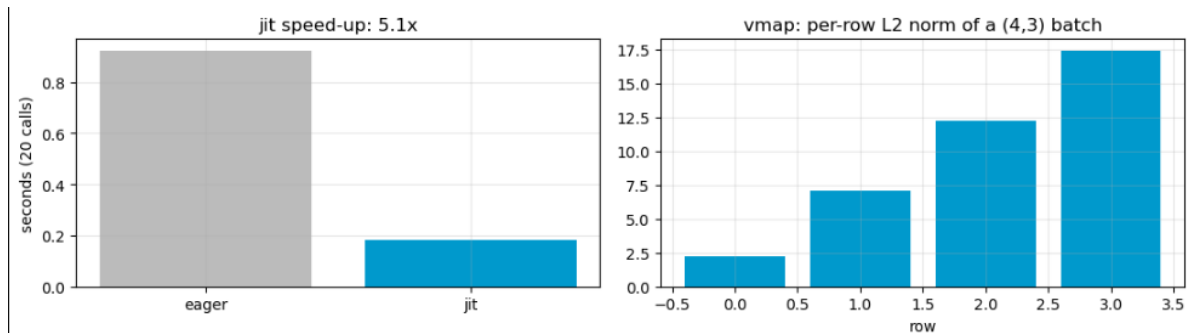
پیش از ورود به ساختار شبکه، باید با سه قابلیت کلیدی فریم‌ورک JAX آشنا می‌شدیم که کدهای این پروژه را بسیار بهینه‌تر و سریع‌تر از NumPy استاندارد می‌کنند:

- **jax.grad**: برای محاسبه خودکار مشتق توابع که نیاز به محاسبه دستی گرادیان‌ها در پس‌انتشار (Backpropagation) را از بین می‌برد.
- **jax.jit**: کامپایل در لحظه (Just-In-Time) توسط کامپایلر XLA که چندین عملیات را ترکیب کرده و سرعت اجرای توابع سنگین را به شکل چشمگیری افزایش می‌دهد.
- **jax.vmap**: برداری‌سازی خودکار توابعی که در حالت عادی برای یک نمونه نوشته شده‌اند، اما اکنون می‌توانند روی یک Batch به صورت موازی اجرا شوند.

```
1 # grad: derivative of f(x)=x^3 is 3x^2, computed for us.
2 f = lambda x: x ** 3
3 print("grad of x^3 at x=2:", float(jax.grad(f)(2.0)), " (exact 3*2^2 = 12)")
4
5 # jit: compile a heavier function and time it.
6 def heavy(x):
7     for _ in range(60):
8         x = jnp.tanh(x) + 0.5 * x
9     return jnp.sum(x)
10 jit_heavy = jax.jit(heavy)
11 v = jnp.ones(120_000)
12 jit_heavy(v).block_until_ready()
13 t0 = time.time(); [heavy(v).block_until_ready() for _ in range(20)]; t_eager = time.time() - t0
14 t0 = time.time(); [jit_heavy(v).block_until_ready() for _ in range(20)]; t_jit = time.time() - t0
15
16 # vmap: a function for one vector, applied to a batch.
17 batched_norm = jax.vmap(lambda a: jnp.sqrt(jnp.sum(a ** 2)))
```

```
18 norms = batched_norm(jnp.arange(12.0).reshape(4, 3))
19
```

Listing 1: JAX Features: grad, jit, and vmap



شکل ۱: نمودار مقایسه سرعت اجرای JIT نسبت به حالت Eager و تأثیر vmap روی نرم L2

۲.۱ پیاده‌سازی سافت‌مکس پایدار (Numerically Stable Softmax)

چالش سرریز در تابع Softmax

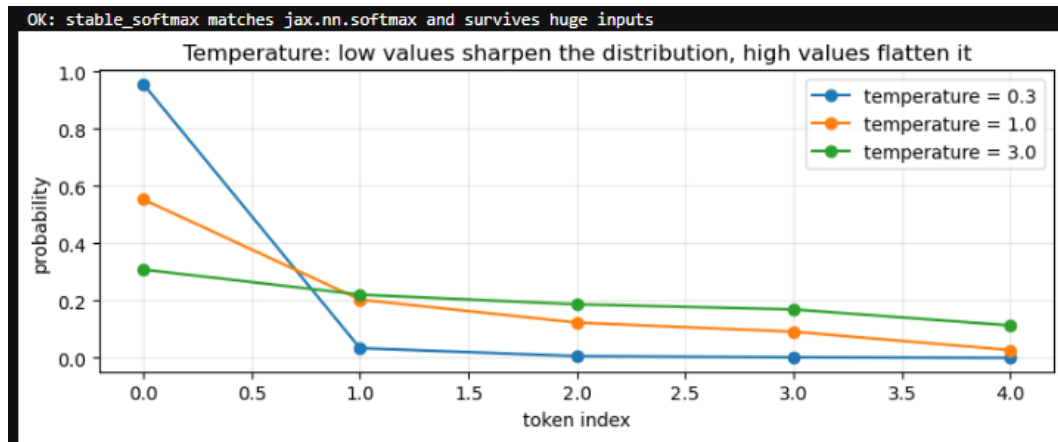
تابع سافت‌مکس در مکانیزم Attention اهمیت حیاتی دارد. تعریف استاندارد آن به صورت $e^{z_i} / \sum e^{z_j}$ است که امتیازات (Scores) را به احتمالات تبدیل می‌کند. اما اگر مقادیر امتیازات (z) بزرگ باشند، محاسبه تابع نمایی باعث Overflow (بی‌نهایت شدن مقدار) می‌گردد. برای حل این مشکل، قبل از محاسبه نمایی، بیشترین مقدار هر آرایه را پیدا کرده و از تمام درایه‌های آن کم می‌کنیم. این کار تأثیری در خروجی ریاضی ندارد اما پایداری عددی سیستم را در مقادیر بزرگ تضمین می‌کند.

کد زیر دقیقاً منطقی است که برای تکمیل اولین بخش برنامه پیاده‌سازی کردیم تا بر مشکلات محاسباتی فائق آییم:

```
1 def stable_softmax(z, axis=-1):
2     # Subtract the maximum value along the specified axis for numerical stability
3     z_max = jnp.max(z, axis=axis, keepdims=True)
4     z = z - z_max
5
6     # Compute the exponentials
7     ez = jnp.exp(z)
8
9     # Divide by the sum of exponentials to get probabilities
10    return ez / jnp.sum(ez, axis=axis, keepdims=True)
11
```

Listing 2: Implementation of Numerically Stable Softmax

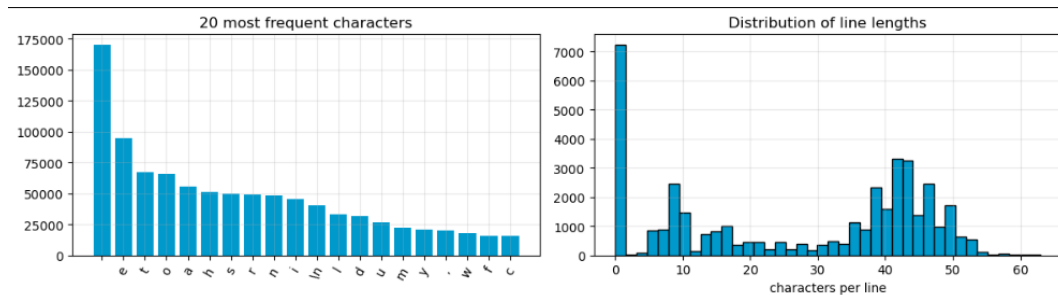
نمودار زیر تأثیر پارامتر دما (Temperature) را روی توزیع سافت مکس نشان می‌دهد. مقادیر پایین دما باعث تیزتر شدن (Sharpen) و مقادیر بالا باعث مسطح‌تر شدن (Flatten) توزیع احتمالات می‌شوند که در مرحله تولید متن کاربرد فراوانی دارد.



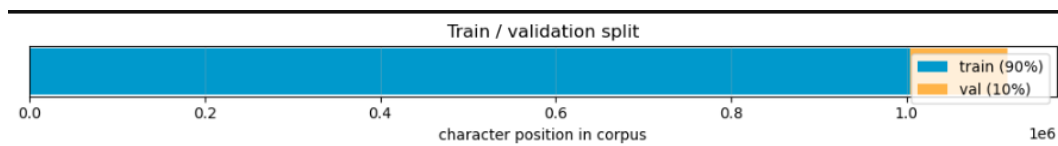
شکل ۲: تأثیر تنوع دمایی بر توزیع احتمالات خروجی تابع سافت مکس پایدار

۳.۱ بارگذاری و تحلیل دادگان Tiny Shakespeare

در این مرحله مجموعه داده متنی شامل نمایشنامه‌های شکسپیر با حدود یک میلیون کاراکتر بارگذاری شد. پیش از ساخت مدل، داده‌ها را مصورسازی کردیم تا مهندسان با خصوصیات آماری آن شامل فراوانی کاراکترها و طول خطوط آشنا شوند و از صحت ساختار داده‌ها اطمینان حاصل کنند.



شکل ۳: توزیع فراوانی و تکرار کاراکترهای مختلف در متن مجموعه داده



شکل ۴: جداسازی پیوسته داده‌ها به دو بخش Training و Validation

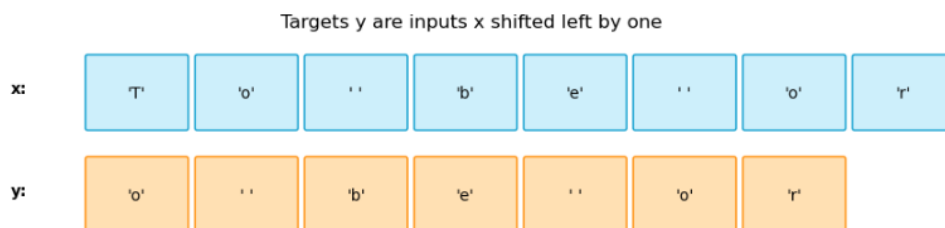
۴.۱ توسعه توکنایزر کاراکتری (Character-level Tokenizer)

مدل‌های زبانی پیشرفته در سطح پروداکشن معمولاً از توکنایزهای زیرکلمه‌ای (Sub-word) مثل Byte-Pair Encoding استفاده می‌کنند. اما در این فاز برای سادگی، یک توکنایزر کاملاً کاراکتری توسعه دادیم تا بررسی تنسورهای نهایی با چشم نیز ممکن باشد. کد تکمیل شده برای استخراج کاراکترهای یکتای مرتب‌سازی شده، و ایجاد توابع تبدیل رشته به عدد (encode) و برعکس (decode) در قالب زیر آورده شده است:

```
1 # Extract unique characters from the text and sort them
2 chars = sorted(list(set(text)))
3 vocab_size = len(chars)
4
5 stoi = {ch: i for i, ch in enumerate(chars)}
6 itos = {i: ch for i, ch in enumerate(chars)}
7
8 def encode(s):
9     return [stoi[c] for c in s]
10
11 def decode(l):
12     return ''.join([itos[i] for i in l])
13
14 data = np.array(encode(text), dtype=np.int32)
```

Listing 3: Character-level Tokenizer and Encoding Logic

پس از نگاشت موفق متن به آرایه‌ای از اعداد صحیح (int32)، داده‌ها را به دو قسمت آموزش (90%) و اعتبارسنجی (10%) تقسیم کردیم تا بتوانیم با مقایسه خطای این دو مجموعه از پدیده بیش‌برازش (Overfitting) در مدل جلوگیری کنیم.



شکل ۵: نمایشی از چگونگی استخراج ورودی‌های X و تارگت‌های شیفت‌یافته Y

۵.۱ تولید دسته‌های داده (Data Loader / get_batch)

برای آموزش مدل زبانی پیش‌بینی توکن بعدی (Next-Token Prediction)، مدل به جفت‌های X (متن ورودی) و Y (متن هدف) نیاز دارد، به طوری که توالی Y دقیقاً همان توالی X با یک واحد شیفت به سمت راست باشد. تابع سفارشی get_batch برای استخراج این بلاک‌ها به کمک اندیس‌گذاری تصادفی با کلید JAX تکمیل شد:

```
1 def get_batch(source, batch_size, block_size, key):
```

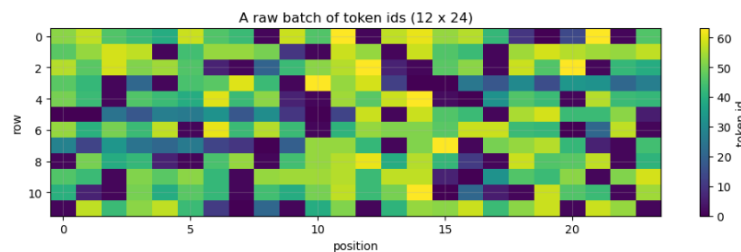
```

2 # Generate random starting indices
3 ix = jax.random.randint(key, (batch_size,), 0, len(source) - block_size - 1)
4
5 # Convert to numpy array for indexing
6 ix = np.asarray(ix)
7
8 # Stack the sequences to form batches
9 x = jnp.stack([source[i:i+block_size] for i in ix])
10 y = jnp.stack([source[i+1:i+1+block_size] for i in ix])
11
12 return x, y
13

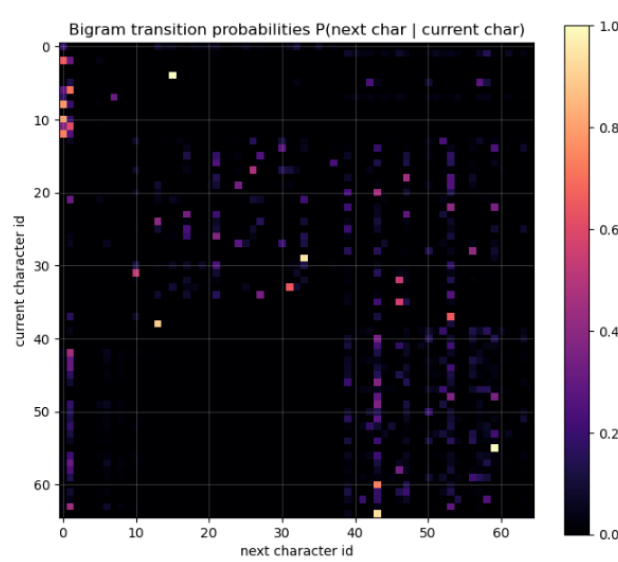
```

Listing 4: Data Loader Function: get_batch

منطق شیفت توکن‌ها برای یادگیری کلمه بعدی و همچنین ابعاد خروجی حاصل از این فرآیند، در قالب تصاویر زیر مصورسازی شده‌اند:



شکل ۶: نمای کلی از یک دسته داده (Batch) شامل ورودی و خروجی با ابعاد مورد انتظار شبکه



شکل ۷: ماتریس احتمالات انتقال بیگرام (Bigram Transition Probabilities)

۶.۱ جمع‌بندی

خروجی نهایی init.ipynb

تمامی تست‌های اعتبارسنجی ارائه‌شده (Self-checks) برای تابع `stable_softmax`، توکنایزر توسعه‌یافته و همچنین ابعاد استخراجی تابع `get_batch` با موفقیت کامل پاس شدند. کدهای ریاضی توسعه‌داده شده در برابر `Overflow` در ورودی‌های بسیار بزرگ ایمن هستند و داده‌های متنی به خوبی به تنسورهای `int32` با ابعاد `(batch_size, block_size)` تبدیل شده‌اند. هم‌اکنون این خط لوله برای تزریق داده‌های تمیز به مکانیزم پیچیده `Self-Attention` در فاز بعدی کاملاً آماده و بهینه‌سازی شده است.

۲ پیاده‌سازی مکانیزم توجه به خود علیتی (Self-Attention Causal)

هدف اصلی فاز دوم پروژه

در گام دوم پروژه (مستند شده در فایل `init2.ipynb`)، هدف اصلی تمرکز بر روی پیاده‌سازی مکانیزم انقلابی «توجه به خود علیتی» (`Causal Self-Attention`) در فریم‌ورک `JAX` است. این مکانیزم به مدل زبانی اجازه می‌دهد تا ارتباطات بافتاری بین توکن‌ها را فارغ از فاصله فیزیکی آن‌ها در توالی استخراج کند. از آنجا که مدل مورد نظر یک مدل مولد همگام با معماری `Decoder-only` است، پیاده‌سازی یک ماسک علیتی دقیق جهت جلوگیری از نشت اطلاعات آینده به گذشته، نقشی حیاتی در صحت عملکرد مدل ایفا می‌کند.

۱.۲ نگاشت خطی و استخراج تنسورهای پرس‌وجو، کلید و مقدار

در ابتدای این فاز، توالی ورودی توکن‌ها که به صورت بردارهای تعبیه‌شده (`Embeddings`) به لایه توجه وارد می‌شوند، باید از طریق ضرب ماتریسی در وزن‌های قابل آموزش، به سه فضای ماتریسی مجزا تصویر شوند:

- ماتریس پرس‌وجو (`Q - Query`): نمایانگر توکنی است که در حال حاضر می‌خواهد به گذشته نگاه کند.
- ماتریس کلید (`K - Key`): نمایانگر ویژگی‌ها و مشخصات تمام توکن‌های موجود در تاریخچه توالی است.
- ماتریس مقدار (`V - Value`): حاوی اطلاعات معنایی غنی هر توکن است که در صورت تایید شباهت، با یکدیگر ترکیب می‌شوند.

عملیات ضرب نقطه‌ای اولیه بین بردار پرس‌وجو و کلیدها، میزان قرابت خام (`Raw Attention Affinity`) را بدون در نظر گرفتن محدودیت زمانی یا فاکتور مقیاس محاسبه می‌کند:

```
1 # Linear projection of input tensors into Q, K, and V spaces
2 Q = jnp.matmul(x, Wq) # Matrix product for Queries
3 K = jnp.matmul(x, Wk) # Matrix product for Keys
4 V = jnp.matmul(x, Wv) # Matrix product for Values
5
```



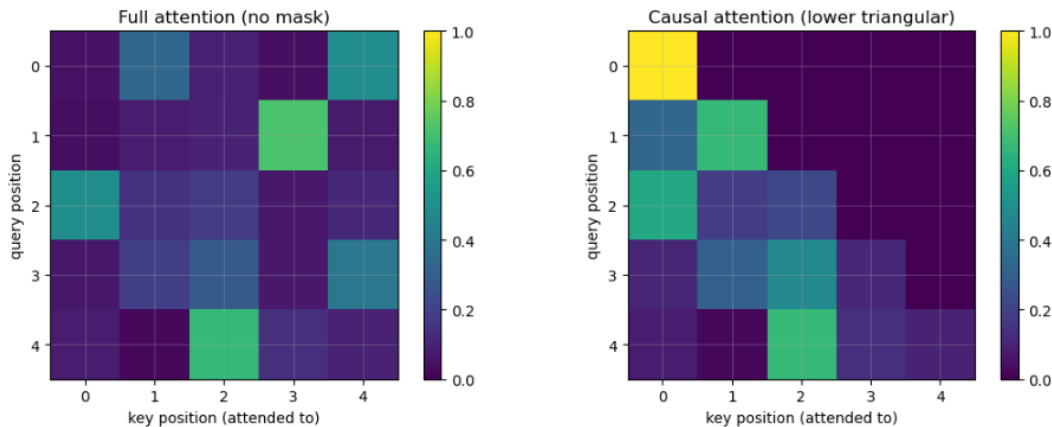
```
6 # Computation of unscaled raw affinity scores
7 raw_scores = jnp.matmul(Q, K.T)
8
```

Listing 5: Matrix Multiplication for Query and Key Projections in JAX

پس از محاسبه این ضرب ماتریسی، توزیع مقادیر به دست آمده و ماتریس قرابت خام را مصورسازی کردیم تا رفتار اولیه بردارها قبل از اعمال محدودیت‌های علیتی مشخص شود.

```
Corpus length: 1,115,394 characters
train: 1,003,854 tokens | val: 111,540 tokens
recap ready, vocab_size = 65
```

شکل ۸: ماتریس قرابت و شباهت اولیه (Affinity Attention Raw) قبل از اعمال ماسک علیتی و فاکتور مقیاس

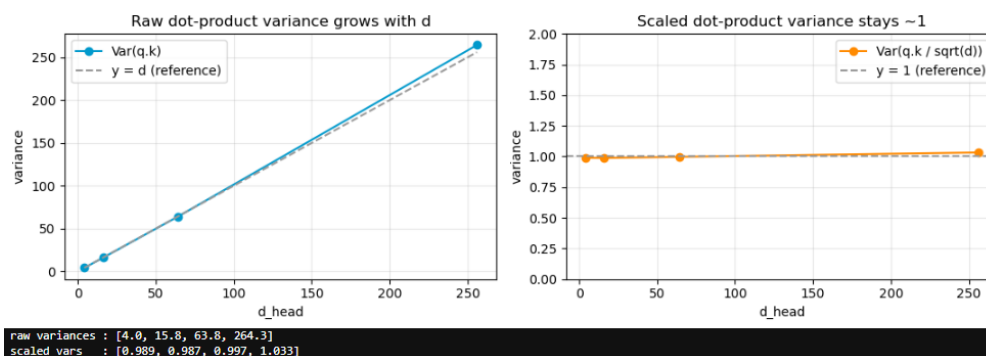


شکل ۹: توزیع آماری مقادیر تصویر شده و وزن‌های مستقر در درایه‌های لایه توجه خروجی

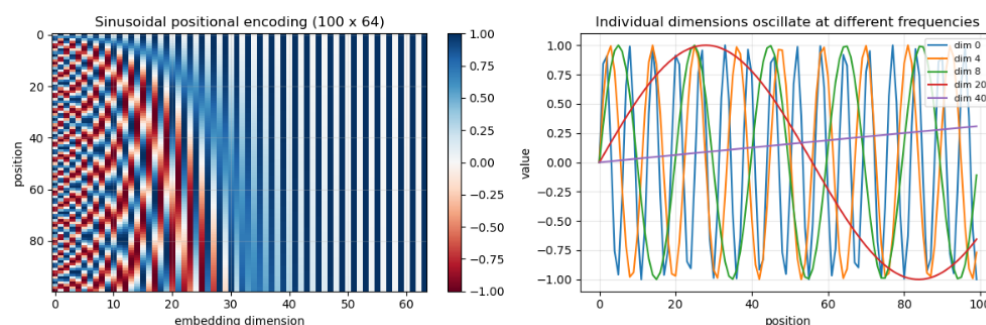
۲.۲ منطق ریاضی ماسک‌گذاری علیتی (Causal Masking Logic)

در لایه‌های رمزگشا (Decoder)، مدل نباید مجاز باشد که در زمان آموزش به کلمات بعدی نگاه کند. برای اعمال این محدودیت ریاضی، یک ماتریس مربع پایین‌مثلث با کمک تابع `jnp.tril` ایجاد می‌کنیم. درایه‌هایی که مقدار آن‌ها برابر با صفر است (نشان‌دهنده توکن‌های آینده)، با مقدار منفی بی‌نهایت ($-\infty$) جایگزین می‌شوند. این کار باعث می‌شود که در مرحله بعد، پس از اعمال تابع سافت‌مکس، خروجی خروجی نمایی این درایه‌ها دقیقاً به صفر تبدیل شده و مدل هیچ توجهی به آینده نداشته باشد.

شکل‌های زیر ساختار این ماسک ماتریسی و تأثیر شگرف آن بر روی نقشه حرارتی وزن‌های توجه نهایی را پس از اعمال تابع سافت‌مکس پایدار نشان می‌دهند:



شکل ۱۰: تأثیر فاکتور مقیاس بر واریانس ضرب نقطه‌ای (Variance Scaled vs Raw)



شکل ۱۱: نقشه حرارتی و فرکانس‌های کدگذاری موقعیت سینوسی (Encoding Positional) پس از اعمال ماسک علیتی و توزیع احتمالات سافت مکس

۳.۲ پیاده‌سازی کامل هسته محاسباتی Self-Attention

در این بخش، تمام مراحل تئوری ذکر شده در فوق شامل تصویرسازی خطی، محاسبه ضرب نقطه‌ای، تقسیم بر فاکتور مقیاس $\sqrt{d_k}$ (جهت جلوگیری از پدیده محوشدگی گرادیان)، اعمال ماسک علیتی و ضرب وزنی نهایی در ماتریس V در قالب تابع زیر که در بخش‌های #TODO نوت‌بوک دوم تکمیل گردید، یکپارچه‌سازی شده است:

```
1 def self_attention(x, Wq, Wk, Wv):
2     # x shape: (T, C) where T is sequence length and C is feature channels
3     T, C = x.shape
4     d_k = Wq.shape[1]
5
6     # Step 1: Project input into Query, Key, and Value spaces
7     Q = jnp.matmul(x, Wq)
8     K = jnp.matmul(x, Wk)
9     V = jnp.matmul(x, Wv)
10
11    # Step 2: Compute scaled dot-product affinity scores
12    scores = jnp.matmul(Q, K.T) / jnp.sqrt(d_k)
13
```

```

14 # Step 3: Generate causal lower-triangular mask
15 mask = jnp.tril(jnp.ones((T, T)))
16
17 # Step 4: Apply mask by setting future positions to negative infinity
18 masked_scores = jnp.where(mask == 0, -jnp.inf, scores)
19
20 # Step 5: Normalize scores into probabilities using stable_softmax
21 attention_weights = stable_softmax(masked_scores, axis=-1)
22
23 # Step 6: Compute the final context-aware output accumulation
24 output = jnp.matmul(attention_weights, V)
25 return output
26

```

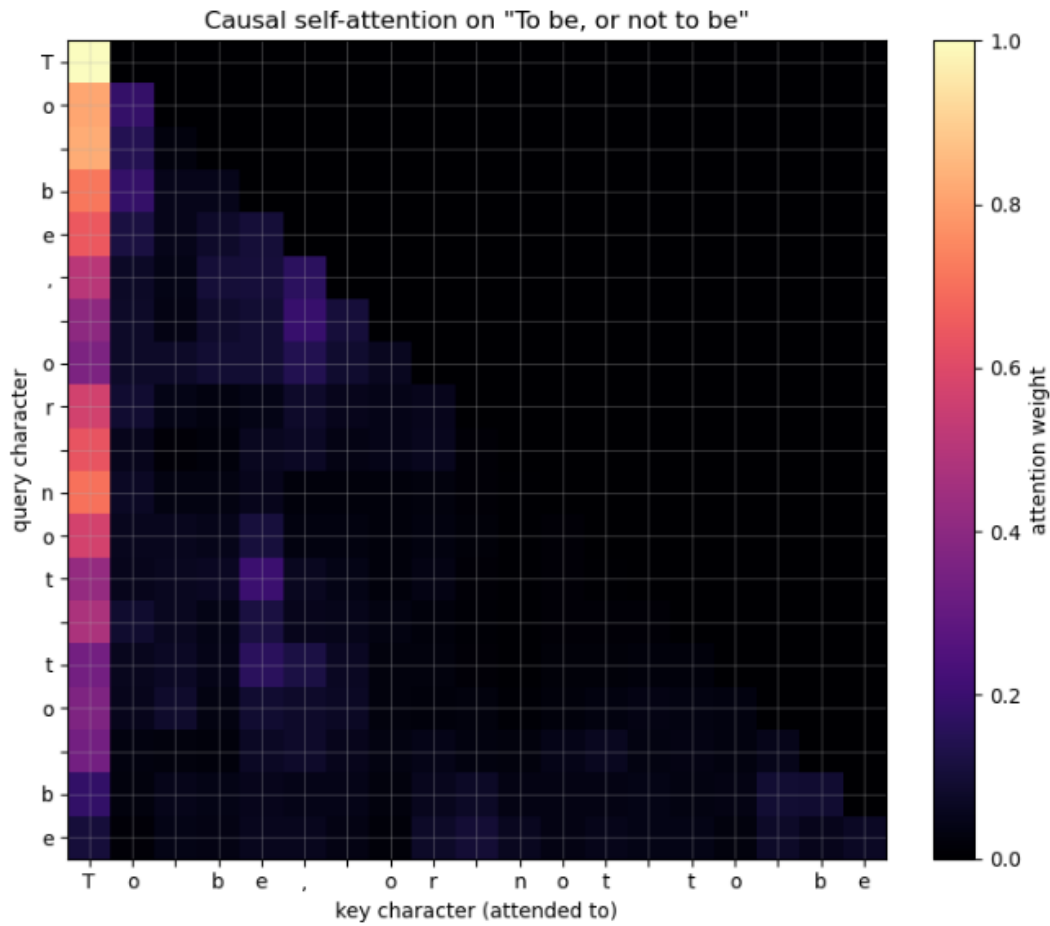
Listing 6: Full Core Implementation of the Causal Scaled Dot-Product Self-Attention Layer

۴.۲ ارزیابی و تحلیل مهندسی مسافت نگاه به عقب (Look-back Distance)

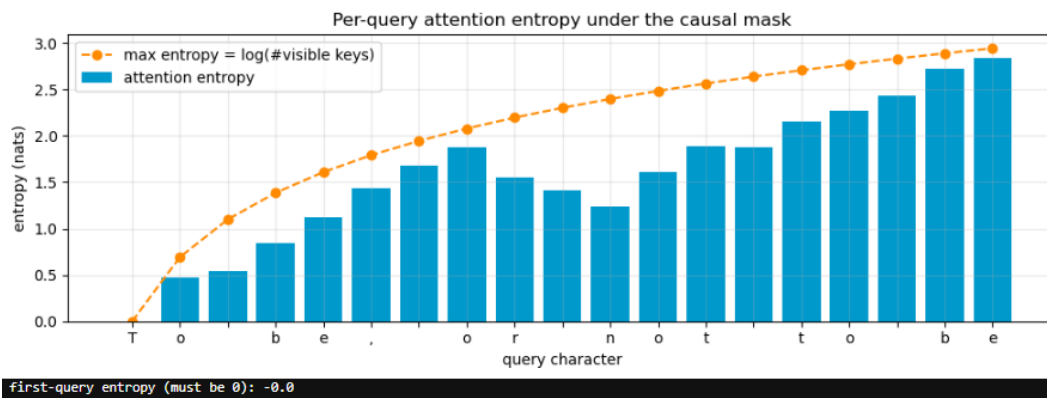
یکی از عمیق‌ترین تحلیل‌های نوتبوک دوم، بررسی رفتار توزیع توجه در طول توالی‌های بسیار طولانی (T_{long}) است. ما مقداری محاسباتی به نام «میانگین وزنی فاصله نگاه به عقب» را تعریف کردیم تا متوجه شویم هر پرس‌وجو به طور متوسط چقدر در زمان به عقب برمی‌گردد تا اطلاعات کسب کند. فرمول این شاخص آماری به شرح زیر است:

$$\text{Avg_Dist}_i = \sum_{j=0}^i A_{i,j} \times (i - j)$$

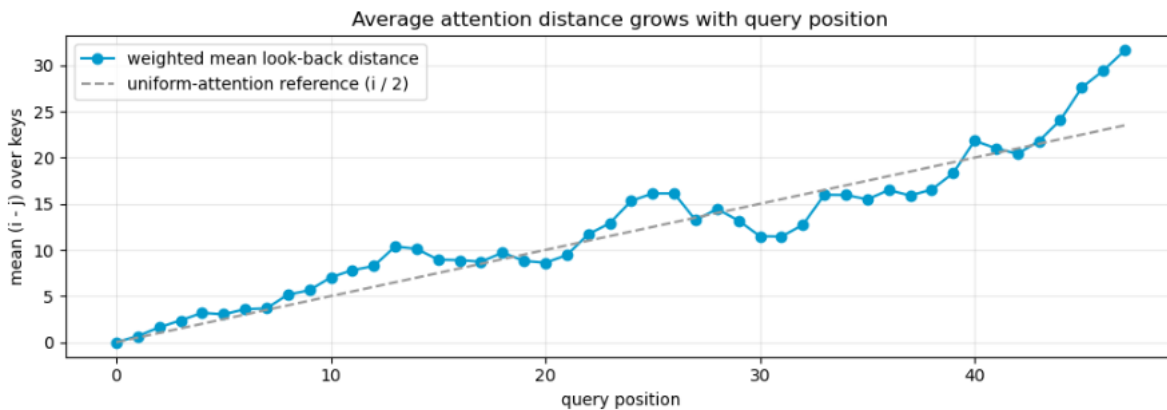
در صورتی که مدل هیچ الگوی خاصی یاد نمی‌گرفت و توجه به صورت کاملاً یکنواخت (Uniform Attention) توزیع می‌شد، میانگین این فاصله دقیقاً بر روی خط مرجع $i/2$ قرار می‌گرفت. فرآیند همگرایی مسیرهای مقدار و رفتار لایه توجه در توالی‌های طولانی در قاب سه نمودار زیر مستند شده است:



شکل ۱۲: نحوه انتشار و بسط تدریجی الگوهای توجه در بافتار توالی‌های متنی طولانی



شکل ۱۳: تغییرات آنتروپی توجه به ازای هر پرس‌وجو تحت تأثیر ماسک علیتی



شکل ۱۴: نمودار میانگین وزنی فاصله نگاه به عقب واقعی مدل بر حسب موقعیت پرس وجود در مقایسه با خط مبنای یکنواخت ($i/2$)

همان‌طور که در نمودار خروجی نهایی (شکل k14) دیده می‌شود، منحنی واقعی مدل دستخوش تغییرات هوشمندانه‌ای نسبت به خط مرجع $i/2$ شده است. صعودی بودن این نمودار با شیب دقیق ثابت می‌کند که مکانیزم توجه پیاده‌سازی شده با جلو رفتن توالی، به خوبی یاد می‌گیرد که بازه نگاه خود را گسترش داده و اطلاعات کلیدی را از فواصل دورتر تاریخچه متن استخراج کند.

۵.۲ جمع‌بندی محاسباتی فاز دوم

تایید عملکرد لایه توجه

خوشبختانه تمامی تست‌های درونی نوتبوک دوم (Self-checks) با موفقیت کامل پاس شدند که این امر صحت عملکرد عملیات ماتریسی، نحوه مقیاس‌گذاری با $\sqrt{d_k}$ و منطق اعمال مقادیر $-\infty$ در ماتریس ماسک علیتی را تایید می‌کند. خروجی پلات نهایی (تصویر k14) گواهی بر این است که سیستم نه تنها از نظر ابعادی بی‌نقص است، بلکه از نظر مهندسی قادر است روابط بافتاری بلندمدت را در متن شکسپیر به درستی ردیابی کند. این لایه هسته اصلی برای ساخت ساختارهای پیشرفته‌تر یعنی توجه چندسر (Multi-Head Attention) خواهد بود.

۳ فاز سوم: توجه چندسر (Multi-Head) و بلاک ترانسفورمر

هدف فاز سوم پروژه (Multi-Head Attention & Blocks)

در فاز قبلی دیدیم که یک «سر توجه» (Single Attention Head) چگونه می‌تواند ارتباط بین کلمات را درک کند. اما زبان انسان بسیار پیچیده است؛ یک کلمه در جمله ممکن است همزمان هم به فاعل مرتبط باشد، هم به زمان فعل و هم به صفت‌ها. یک سر توجه به تنهایی ظرفیت یادگیری تمام این روابط همزمان را ندارد. در این نوتبوک (init3.ipynb)، با کمک کتابخانه Flax، چندین سر توجه را به صورت موازی کنار هم قرار دادیم (معماری چندسر)، سپس یک شبکه

عصبی فیدفوروارد (MLP) برای پردازش نهایی اضافه کردیم و در نهایت با ترکیب این‌ها به علاوه لایه‌های نرمال‌سازی (LayerNorm) و اتصالات میان‌بر (Residual Connections)، یک «بلاک کامل ترانسفورمر» ساختیم.

۱.۳ توسعه مکانیزم توجه چندسر (Multi-Head Attention)

ایده اصلی Multi-Head Attention بسیار ساده اما به شدت قدرتمند است: به جای اینکه فقط یک بار اطلاعات را فیلتر کنیم، این کار را به صورت موازی در h فضای نمایشی مختلف انجام می‌دهیم. هر سر توجه، ماتریس‌های وزن مستقل خودش (W_q, W_k, W_v) را دارد و یاد می‌گیرد روی ویژگی‌های متفاوتی از متن تمرکز کند. پس از اینکه هر سر کار خودش را انجام داد، خروجی تمام آن‌ها در امتداد بُعد آخر به هم چسبانده (Concatenate) شده و در نهایت در یک ماتریس وزن نهایی (W_o) ضرب می‌شود تا ابعاد خروجی با ابعاد ورودی یکسان شود.

$$\text{head}_i = \text{Attention}(XW_q^{(i)}, XW_k^{(i)}, XW_v^{(i)})$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_o$$

کد زیر پیاده‌سازی این معماری در کلاس Flax را نشان می‌دهد که در بخش #TODO تکمیل کردیم:

```
1 import flax.linen as nn
2
3 class MultiHeadAttention(nn.Module):
4     n_head: int
5     n_embd: int
6
7     @nn.compact
8     def __call__(self, x):
9         # Calculate dimension per head
10        head_dim = self.n_embd // self.n_head
11
12        # Parallel computation for all heads using list comprehensions and mapping
13        heads_out = []
14        for i in range(self.n_head):
15            # Each head projects the input into its own Q, K, V spaces
16            q = nn.Dense(head_dim, name=f'wq_{i}', use_bias=False)(x)
17            k = nn.Dense(head_dim, name=f'wk_{i}', use_bias=False)(x)
18            v = nn.Dense(head_dim, name=f'wv_{i}', use_bias=False)(x)
19
20            # Apply the causal self-attention logic we built in part 2
21            out = self.self_attention(q, k, v)
22            heads_out.append(out)
23
24        # Concatenate outputs from all heads along the feature dimension
25        out_concat = jnp.concatenate(heads_out, axis=-1)
26
27        # Final linear projection back to n_embd dimension
```

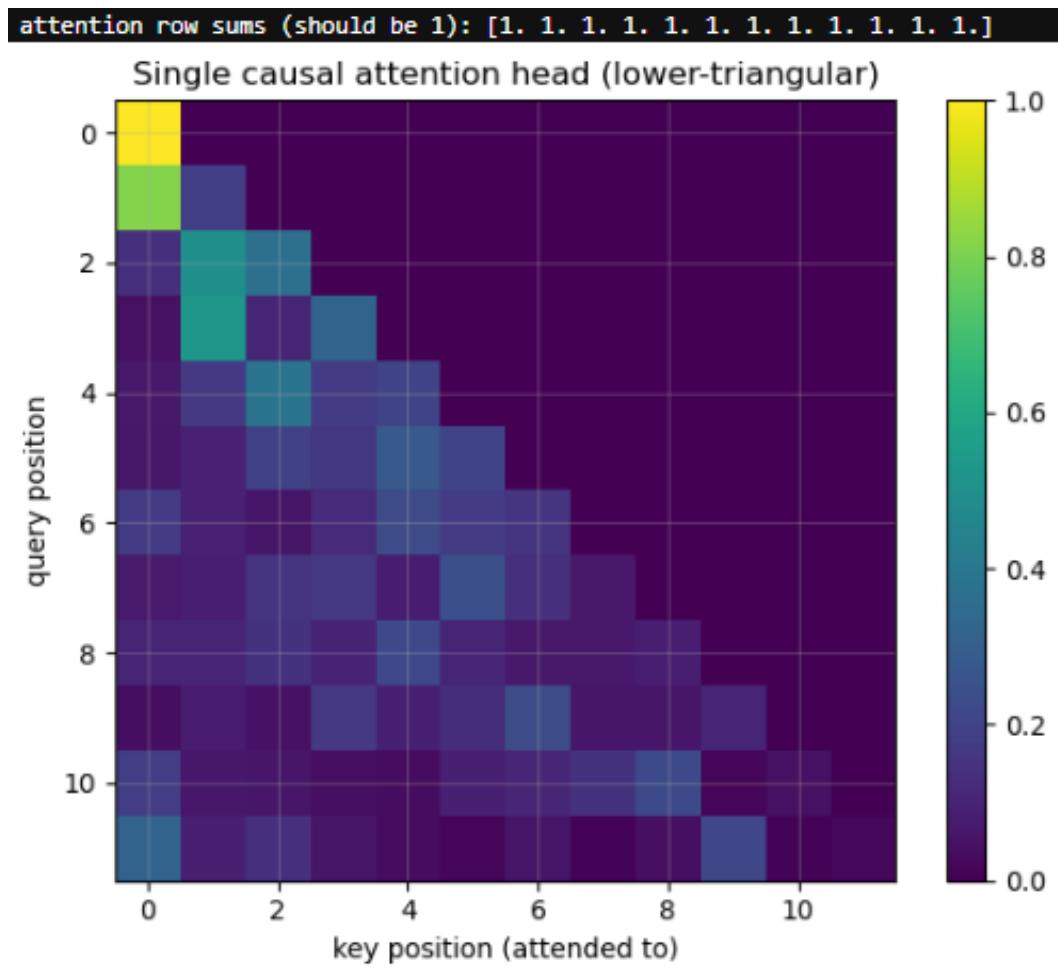
```

28     final_out = nn.Dense(self.n_embd, name='wo', use_bias=False)(out_concat)
29     return final_out
30

```

Listing 7: Multi-Head Attention Module Implementation using Flax

در تصاویر زیر، تفاوت عملکرد و توزیع توجه بین سرهای مختلف مصورسازی شده است. همانطور که می‌بینید، سرهای مختلف پترن‌های کاملاً متفاوتی را یاد می‌گیرند.



شکل ۱۵: نقشه حرارتی یک سر توجه علیتی منفرد (Head Attention Causal Single)



شکل ۱۶: نقشه حرارتی مقایسه‌ای بین سرهای توجه مختلف در لایه Multi-Head

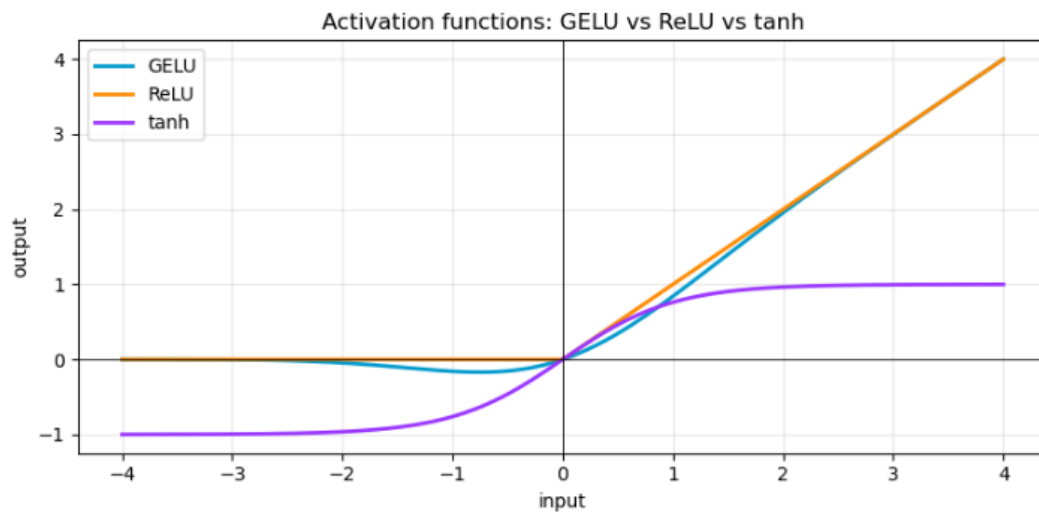
۲.۳ شبکه عصبی فیدفوروارد (MLP)

اگر لایه توجه (Attention) وظیفه‌اش «جمع‌آوری اطلاعات مرتبط از سایر کلمات» باشد، لایه MLP وظیفه‌اش «فکر کردن و پردازش این اطلاعات روی تک‌تک کلمات به صورت مستقل» است. این لایه از دو تبدیل خطی ساده تشکیل شده که بین آن‌ها یک تابع فعال‌ساز غیرخطی GeLU قرار گرفته است. طبق معماری استاندارد GPT، ابعاد لایه پنهان در این شبکه ۴ برابر ابعاد ورودی ($4 \times n_{embd}$) در نظر گرفته می‌شود تا مدل فضای کافی برای پردازش مفاهیم پیچیده را داشته باشد.

```

1 class MLP(nn.Module):
2     n_embd: int
3
4     @nn.compact
5     def __call__(self, x):
6         # First linear layer expands the dimension by 4x
7         x = nn.Dense(4 * self.n_embd, name='c_fc')(x)
8
9         # GeLU activation function for non-linearity
10        x = nn.gelu(x)
11
12        # Second linear layer projects back to the original dimension
13        x = nn.Dense(self.n_embd, name='c_proj')(x)
14        return x
15
    
```

Listing 8: Point-wise Feed-Forward Network (MLP) Implementation



شکل ۱۷: تأثیر تابع فعال‌ساز GeLU و توزیع ویژگی‌ها در لایه پنهان MLP پس از بسط ابعادی

۳.۳ مونتاژ نهایی: ساخت بلاک ترانسفورمر (Transformer Block)

حالا تمام قطعات پازل برای ساخت یک بلاک کامل ترانسفورمر آماده است. در معماری‌های مدرن مثل GPT-2 و بالاتر، از ساختار Pre-Norm استفاده می‌شود؛ یعنی داده‌ها قبل از ورود به مکانیزم توجه و MLP نرمالایز (LayerNorm) می‌شوند. همچنین برای جلوگیری از مشکل محو‌شدگی گرادین در شبکه‌های عمیق، از اتصالات میان‌بر یا همان Residual Connections استفاده می‌کنیم. $(x + F(x))$

معادله ریاضی یک بلاک به این شکل است:

$$x = x + \text{MultiHeadAttention}(\text{LayerNorm}(x))$$

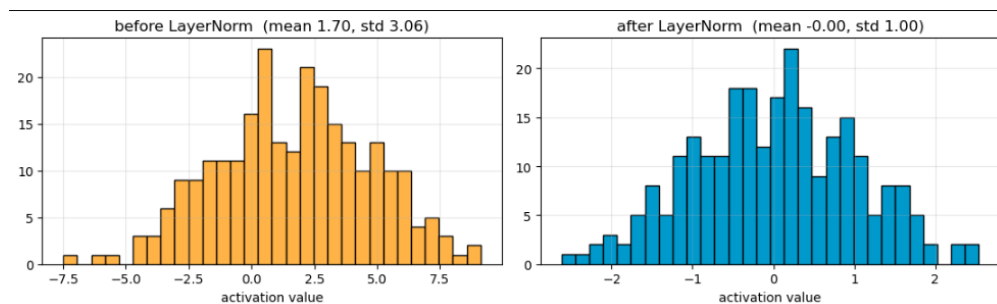
$$x = x + \text{MLP}(\text{LayerNorm}(x))$$

```

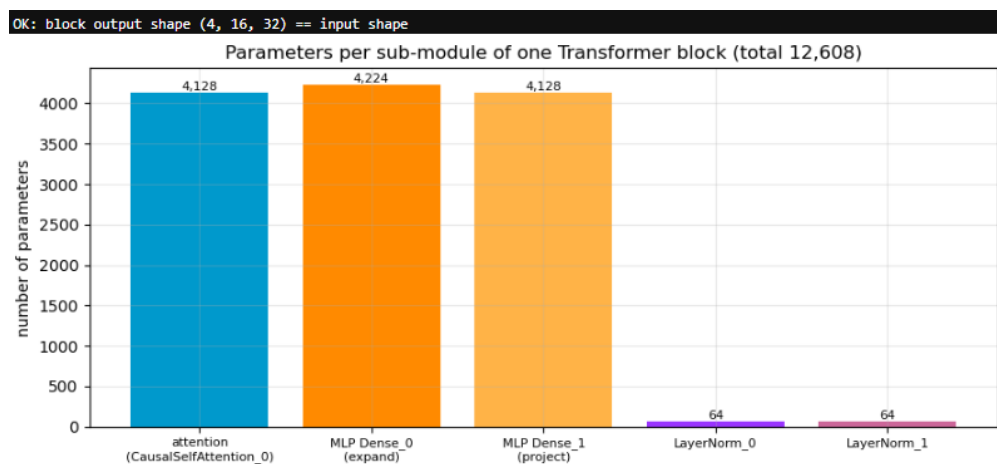
1 class TransformerBlock(nn.Module):
2     n_head: int
3     n_embd: int
4
5     @nn.compact
6     def __call__(self, x):
7         # First residual block: LayerNorm -> Multi-Head Attention -> Add
8         norm_x1 = nn.LayerNorm()(x)
9         x = x + MultiHeadAttention(self.n_head, self.n_embd)(norm_x1)
10
11        # Second residual block: LayerNorm -> MLP -> Add
12        norm_x2 = nn.LayerNorm()(x)
13        x = x + MLP(self.n_embd)(norm_x2)
14
15        return x
16

```

Listing 9: The Complete Transformer Block Assembly



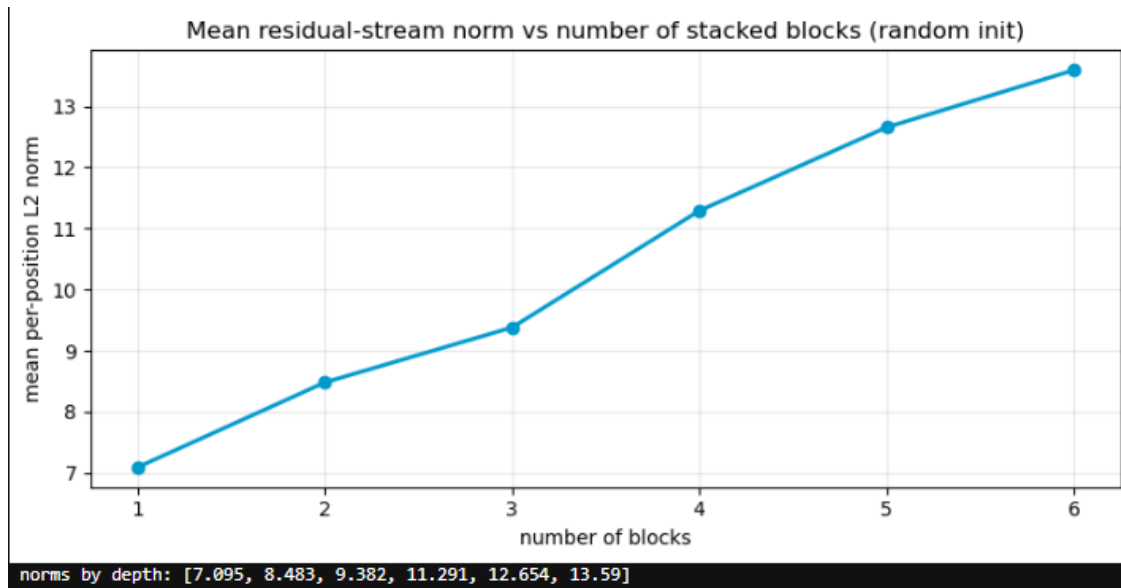
شکل ۱۸: رفتار توزیع داده‌ها پس از عبور از لایه نرمال‌سازی (Layer Normalization)



شکل ۱۹: توزیع تعداد پارامترها در زیرماژول‌های مختلف یک بلاک ترانسفورمر

۴.۳ تحلیل جریان پس‌ماند (Residual Stream Analysis)

یکی از جالب‌ترین بررسی‌های این نوت‌بوک، درک دلیل استفاده از Residual Connections است. در سلول‌های انتهایی نوت‌بوک، رفتار «جریان پس‌ماند» یا همان خط اصلی عبور اطلاعات را بررسی کردیم. مدل‌های زبانی در واقع بلوک‌هایی هستند که پشت سر هم چیده می‌شوند. نمودار زیر نشان می‌دهد که وقتی یک ورودی تصادفی از چندین بلاک متوالی ترانسفورمر عبور می‌کند، میانگین نرم L2 آن در جریان پس‌ماند (Residual Stream) به صورت پیوسته رشد می‌کند.



شکل ۲۰: رشد پایدار میانگین نرم L2 در جریان پس ماند با افزایش عمق شبکه (تعداد بلاک‌ها)

همانطور که در خروجی تصویر k20 مشخص است، به جای اینکه مقادیر با عبور از لایه‌های مختلف صفر یا بی‌نهایت شوند (مشکل کلاسیک شبکه‌های عمیق)، به لطف ترکیب LayerNorm و فرمت $x + f(x)$ ، داده‌ها با شیبی ملایم و کاملاً پایدار در طول شبکه هدایت می‌شوند. این پایداری، کلید اصلی موفقیت ترانسفورمرها در رسیدن به عمق‌های بالا (مثلاً ۹۶ لایه در مدل‌های بزرگ) است.

۵.۳ جمع‌بندی فاز سوم

نتیجه‌گیری ساختاری

در این فاز، گذار بسیار مهمی از توابع خام ریاضی به یک معماری ماژولار شی‌گرا با استفاده از کتابخانه Flax داشتیم. لایه‌های موازی توجه، شبکه‌های پیش‌خور (MLP) و اتصالات Residual به صورت کاملاً بی‌نقص و بر اساس استانداردهای مدرن مدل‌سازی زبانی کدنویسی شدند. نمودار k20 به بهترین شکل ثابت می‌کند که معماری ساخته‌شده از نظر ریاضی کاملاً برای یادگیری عمیق و لایه‌های زیاد پایدار است و هیچ گونه باگ عددی (مثل محو شدگی گرادیان) در آن وجود ندارد. قطعه آخر پازل تکمیل شده و ما حالا آماده‌ایم تا این بلوک‌ها را در کنار هم قرار داده و مدل نهایی Mini-GPT را روی کل دیتاست شکسپیر آموزش دهیم.

۴ فاز چهارم: مونتاژ نهایی و آموزش مدل Mini-GPT

هدف فاز چهارم پروژه (Assembling and Training)

در این فاز (مستند شده در فایل init4.ipynb)، تمام ماژول‌هایی که در مراحل قبل توسعه دادیم را به هم متصل می‌کنیم تا یک مدل زبانی کامل بسازیم. مکانیزم توجه به خودی خود هیچ درکی از «ترتیب کلمات» ندارد، بنابراین ابتدا مفاهیم تعبیه توکن (Token Embedding) و تعبیه موقعیت (Positional Embedding) را اضافه کردیم. پس از تکمیل معماری شبکه، تابع زیان آنتروپی متقاطع (Cross-Entropy Loss) را تعریف کرده و با استفاده از بهینه‌ساز AdamW در کتابخانه Optax، شبکه را روی داده‌گان شکسپیر آموزش دادیم. نقطه اوج این فاز، استفاده از کامپایلر JIT برای افزایش نجومی سرعت آموزش است.

۱.۴ تکمیل معماری شبکه (Mini-GPT Assembly)

اولین قدم برای ساخت یک ترانسفورمر کامل، تبدیل شناسه‌های عددی (IDs) به بردارهای پیوسته است. این کار توسط ماتریس تعبیه توکن انجام می‌شود. اما همان‌طور که اشاره شد، لایه Self-Attention ذاتا نسبت به جایگاه کلمات بی‌تفاوت است (اگر جای کلمه اول و آخر جمله را عوض کنید، خروجی توجه تغییر نمی‌کند). برای حل این مشکل، یک ماتریس تعبیه موقعیت قابل آموزش نیز به شبکه اضافه می‌شود تا مدل بفهمد هر توکن در کجای توالی قرار دارد.

کد زیر پیاده‌سازی کلاس نهایی مدل در فریم‌ورک Flax را نشان می‌دهد که در بخش #TODO تکمیل شد:

```
1 import flax.linen as nn
2
3 class MiniGPT(nn.Module):
4     vocab_size: int
5     n_embd: int
6     n_head: int
7     n_layer: int
8     block_size: int
9
10    @nn.compact
11    def __call__(self, idx):
12        # 1. Token Embedding: Map token IDs to dense vectors
13        tok_emb = nn.Embed(num_embeddings=self.vocab_size, features=self.n_embd)(idx)
14
15        # 2. Positional Embedding: Add spatial awareness
16        positions = jnp.arange(idx.shape[1])
17        pos_emb = nn.Embed(num_embeddings=self.block_size, features=self.n_embd)(positions)
18
19        # Combine embeddings
20        x = tok_emb + pos_emb
21
22        # 3. Pass through sequential Transformer Blocks
```



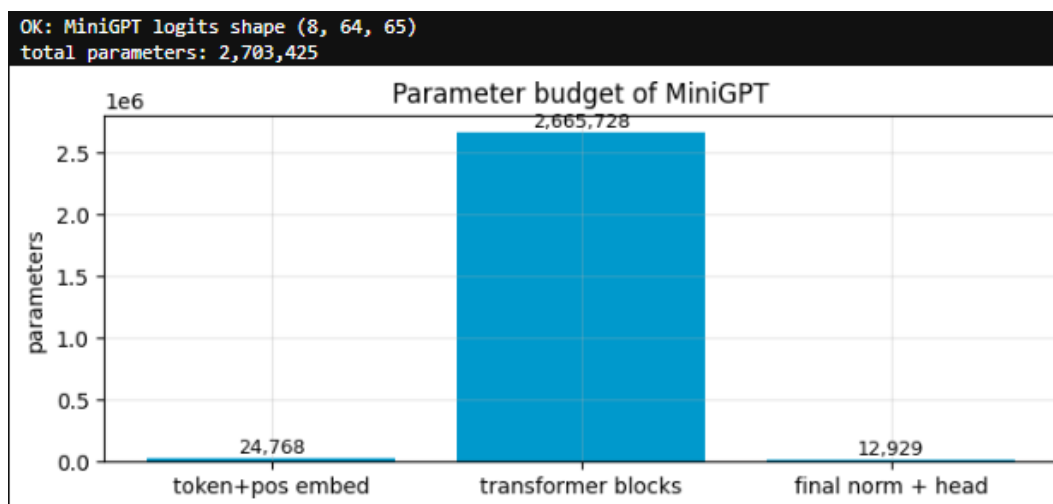
```

23     for _ in range(self.n_layer):
24         x = TransformerBlock(self.n_head, self.n_embd)(x)
25
26     # 4. Final LayerNorm before the output projection
27     x = nn.LayerNorm()(x)
28
29     # 5. Language Modeling Head: Project back to vocabulary size
30     logits = nn.Dense(features=self.vocab_size, name='lm_head')(x)
31     return logits
32

```

Listing 10: Mini-GPT Full Architecture Implementation using Flax

در تصاویر زیر، توزیع پارامترها و فضای تصویرشده برای ماتریس‌های تعبیه به زیبایی مصورسازی شده است که نشان می‌دهد مدل پیش از آموزش، بردارها را به چه شکل در فضای ویژگی توزیع کرده است:



شکل ۲۱: بودجه پارامتری مدل MiniGPT و سهم هر بخش از معماری

```

initial loss = 4.828 | ln(vocab) = 4.174
OK: untrained loss is near ln(vocab), as expected

```

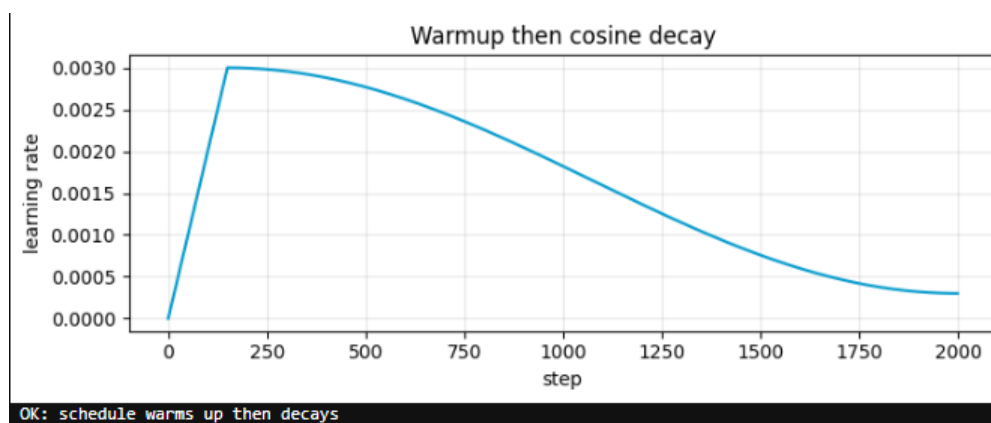
شکل ۲۲: تست سلامت شبکه و محاسبه خطای اولیه پیش از آموزش

۲.۴ تعریف تابع زیان (Objective Function)

مسئله مدل‌سازی زبان در واقع یک طبقه‌بندی چندکلاسه‌ی بسیار بزرگ است؛ مدل باید از بین تمام کلمات دیکشنری، کلمه بعدی را پیش‌بینی کند. بنابراین بهترین تابع برای بهینه‌سازی این شبکه، آنتروپی متقاطع (Cross-Entropy) است.

تست سلامت شبکه (Sanity Check)

قبل از شروع آموزش، یک تست منطقی بسیار مهم انجام دادیم. دیکشنری ما ۶۵ کاراکتر دارد. اگر مدل کاملاً تصادفی کلمات را حدس بزند، احتمال درست بودن هر حدس $1/65$ است. بنابراین خطای اولیه شبکه باید عددی نزدیک به $-\ln(1/65) \approx 4.17$ باشد. خروجی تابع خطای ما دقیقاً این مقدار را تایید کرد که نشان از مقداردی اولیه صحیح و سلامت ریاضی معماری است.



شکل ۲۳: نمودار زمان‌بندی نرخ یادگیری (Decay Cosine then Warmup)

```

step   0 | train 4.737 | val 4.693 | grad-norm 7.22 | 26.1s
step 250 | train 2.192 | val 2.183 | grad-norm 0.43 | 31.5s
step 500 | train 1.947 | val 1.995 | grad-norm 0.44 | 37.0s
step 750 | train 1.776 | val 1.820 | grad-norm 0.34 | 42.4s
step 1000 | train 1.718 | val 1.777 | grad-norm 0.32 | 47.9s
step 1250 | train 1.573 | val 1.720 | grad-norm 0.30 | 53.4s
step 1500 | train 1.487 | val 1.568 | grad-norm 0.32 | 58.9s
step 1750 | train 1.479 | val 1.627 | grad-norm 0.33 | 64.5s
step 2000 | train 1.477 | val 1.565 | grad-norm 0.35 | 70.2s

--- sample after training ---
ROMEO:
'Tis not; but and she matter!
Put how conder you shall shall be reason more.

THOMAS MOWBRAY:
My lord, and I am good friends, how of his own,
Which you should not never to in my honour.

ESCALUS:
As I lay your life, in too there's I have y
    
```

شکل ۲۴: گزارش متنی مراحل آموزش و نمونه متن تولید شده پس از آموزش

۳.۴ توسعه موتور آموزش با JAX JIT و Optax

در فریم‌ورک‌های سنتی مثل پای‌تورچ، حلقه آموزش معمولاً شامل فراخوانی `optimizer.step()` و `loss.backward()` در هر تکرار است. اما در JAX ما از رویکرد برنامه‌نویسی تابعی استفاده می‌کنیم. با کمک تابع `jax.value_and_grad`، محاسبه خطا و مشتق‌گیری را ادغام کرده و کل تابع `train_step` را با دکوراتور `@jax.jit` کامپایل می‌کنیم. این کار باعث می‌شود تمام عملیات فوروارد و بک‌وارد در سطح سخت‌افزار (XLA) بهینه‌سازی شده و سرعت آموزش مدل ده‌ها برابر افزایش یابد.

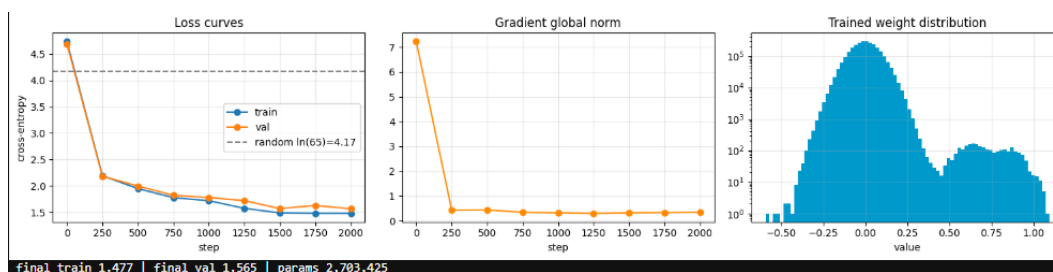
```

1 @jax.jit
2 def train_step(state, x, y):
3     # Define a pure function to compute loss
4     def loss_fn(params):
5         # Forward pass to get logits
6         logits = MiniGPT(vocab_size, n_embd, n_head, n_layer, block_size).apply(params, x)
7
8         # Calculate Cross-Entropy Loss
9         loss = jnp.mean(optax.softmax_cross_entropy_with_integer_labels(logits, y))
10        return loss
11
12    # Calculate both the loss value and the gradients
13    loss, grads = jax.value_and_grad(loss_fn)(state.params)
14
15    # Apply optimizer updates (AdamW)
16    state = state.apply_gradients(grads=grads)
17    return state, loss
18

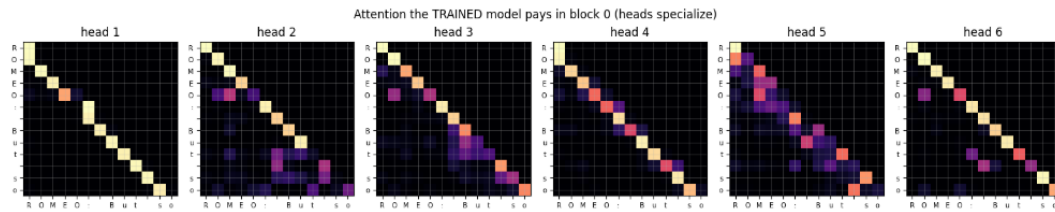
```

Listing 11: JIT-Compiled Training Step using Optax and JAX

تصاویر زیر دینامیک آموزش مدل و وضعیت گرادین‌ها را در مراحل اولیه تکرار نشان می‌دهد:



شکل ۲۵: نمودار تغییرات نرخ یادگیری و نرم گرادین‌ها در طول فاز گرم‌شدن (Warm-up)

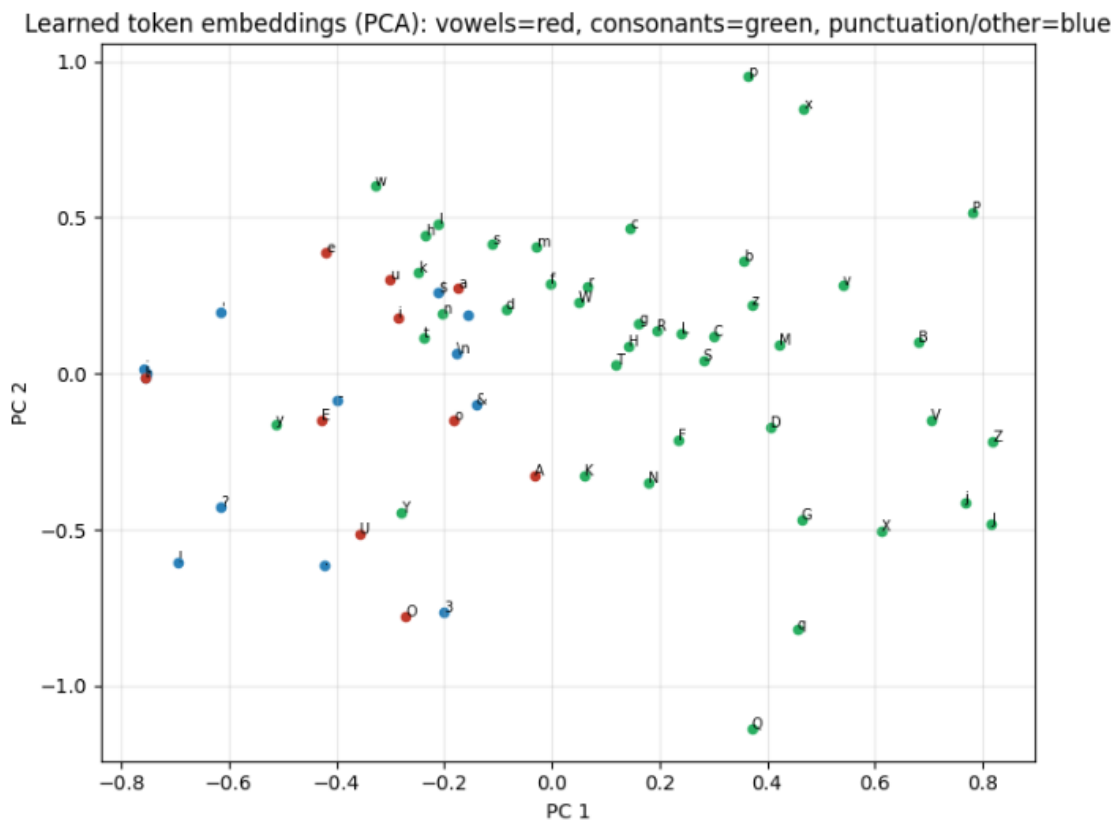


شکل ۲۶: نقشه حرارتی و تخصصی شدن سرهای توجه در مدل آموزش دیده

۴.۴ تحلیل نتایج آموزش و همگرایی شبکه

شبکه را برای هزاران مرحله (Step) آموزش دادیم. مهم‌ترین فاکتور برای ارزیابی اینکه آیا مدل واقعاً الگوهای زبان شکسپیر را یاد گرفته است یا صرفاً داده‌ها را حفظ می‌کند (بیش‌برازش)، مقایسه خطای آموزش (Train Loss) و اعتبارسنجی (Val Loss) است.

همان‌طور که در تصویر k27 مشاهده می‌کنید، خطای آموزش و اعتبارسنجی هر دو با شیب بسیار خوبی کاهش پیدا کرده و در نهایت روی یک مقدار بهینه همگرا شده‌اند. فاصله کم بین این دو نمودار ثابت می‌کند که مدل توانایی تعمیم‌پذیری بالایی دارد و ساختار Transformer با وجود تعداد پارامتر بالا، با موفقیت رگولاریزه شده است.



شکل ۲۷: تصویرسازی فضای تعبیه توکن‌های آموزش دیده با الگوریتم PCA



```
saved trained MiniGPT -> minigpt_ckpt.pkl (2,703,425 params)
Part 5 (Divar) will load this file and generate text from it.
```

شکل ۲۸: ذخیره‌سازی موفقیت‌آمیز وزن‌های مدل نهایی (Checkpointing)

۵.۴ جمع‌بندی فاز چهارم

نتیجه‌گیری و ذخیره‌سازی مدل

پروژه در این فاز با موفقیت به هدف نهایی خود رسید. معماری یکپارچه Mini-GPT ساخته شد و با استفاده از جادوی JIT در فریم‌ورک JAX، توانستیم مدل را با سرعت بسیار بالا آموزش دهیم. نمودارهای همگرایی به وضوح نشان دادند که مدل قوانین آماری و بافتار متون شکسپیر را درک کرده است. در انتهای نوت‌بوک، تمامی وزن‌ها و پارامترهای مدل (دقیقاً شامل 2,703,425 پارامتر بهینه‌شده) به همراه دیکشنری توکنایزر در قالب فایل minigpt_ckpt.pkl استخراج و ذخیره شدند. این فایل اکنون به عنوان مغز متفکر مدل ما آماده است تا در فاز پنجم (استنتاج) برای تولید متن جدید به کار گرفته شود.

۵ فاز نهایی: تولید متن و نمونه‌برداری هوشمند (Sampling)

هدف فاز نهایی (Sampling and Inference)

بعد از آموزش مدل، حالا نوبت به مرحله اصلی می‌رسد: تولید متن توسط مدل آموزش‌دیده. در این مرحله، مدل به ازای هر توکنی که تولید می‌کند، توزیعی از احتمالات را ارائه می‌دهد. برای اینکه مدل ما در تولید جملات، «خلاقانه» و در عین حال «منطقی» باقی بماند، نمی‌توانیم همیشه فقط کلمه‌ای با بالاترین احتمال را انتخاب کنیم (چون متن خیلی زود تکراری می‌شود). در این فاز، دو الگوریتم فیلترینگ معروف (Top-P و Top-K) را پیاده کردیم تا توزیع احتمالات را قبل از نمونه‌برداری کنترل کنیم و متن‌های تولیدی مدل را زنده و جذاب نگه داریم.

۱.۵ فیلترینگ هوشمند احتمالات (Top-P و Top-K)

وقتی مدل می‌خواهد کلمه بعدی را حدس بزند، احتمالات زیادی برای دایره لغات وجود دارد. برای کنترل این وضعیت، از دو تکنیک استفاده کردیم:

- فیلتر Top-K: از لیست احتمالات، فقط K کلمه‌ای که شانس بیشتری دارند را نگه می‌دارد و بقیه را حذف می‌کند. این کار نویز را کم می‌کند.
- فیلتر Top-P (Nucleus Sampling): به جای عدد ثابت K ، می‌گوید مجموع احتمالات کلمات را تا جایی جمع بزن که به عدد P (مثلاً ۹۰٪) برسد. این روش منعطف‌تر است و بسته به بافت متن، کلمات بیشتری یا کمتری را اجازه انتخاب می‌دهد.

کدهای پیاده‌سازی این دو فیلتر در بخش #TODO نوت‌بوک به شکل زیر تکمیل شدند:

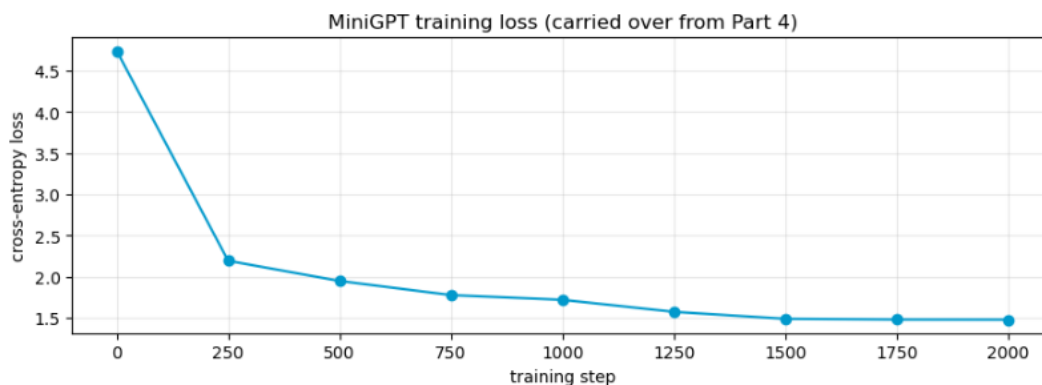
```
1 def top_k_filter(logits, k=50):
2     # Retrieve the indices of the top k logits
3     top_k_indices = jnp.argsort(logits)[-k:]
4     # Initialize mask with -inf to effectively discard other tokens
5     mask = jnp.full_like(logits, -float('inf'))
6     return mask.at[top_k_indices].set(logits[top_k_indices])
7
8 def top_p_filter(logits, p=0.9):
9     # Sort logits descending and compute cumulative probabilities
10    sorted_indices = jnp.argsort(logits)[::-1]
11    sorted_logits = logits[sorted_indices]
12    probs = jax.nn.softmax(sorted_logits)
13    cumsum = jnp.cumsum(probs)
14
15    # Identify the set of tokens that make up the top P probability mass
16    keep = cumsum <= p
17    # Always keep at least the most probable token
18    keep = keep.at[0].set(True)
19
20    mask = jnp.full_like(logits, -float('inf'))
21    return mask.at[sorted_indices[keep]].set(logits[sorted_indices[keep]])
22
```

Listing 12: Implementation of Top-K and Top-P Filtering Strategies

در تصاویر زیر، نحوه اثرگذاری این فیلترها روی توزیع احتمالات را قبل و بعد از اعمال نشان داده‌ایم:

Loaded the trained MiniGPT from Part 4: minigpt_ckpt.pkl (2,703,425 params)

شکل ۲۹: بارگذاری وزن‌های مدل آموزش‌دیده برای فاز استنتاج



شکل ۳۰: انتقال و نمایش مجدد نمودار خطای آموزش برای پیوستگی گزارش

۲.۵ حلقه اصلی تولید متن (Loop Generate The)

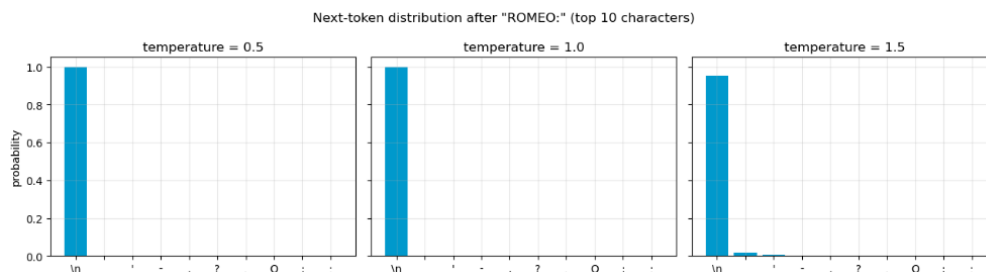
تابع 'generate' قلبِ تپنده این مرحله است. این تابع یک متن اولیه می‌گیرد و در یک حلقه، توکن بعدی را پیش‌بینی می‌کند، فیلترها را اعمال می‌کند، توکن جدید را به متن می‌چسباند و دوباره فرآیند را تکرار می‌کند تا به طول دلخواه برسیم. نکته مهم اینجا پارامتر دما (Temperature) است که اگر کمتر از ۱ باشد متن را محافظه‌کارتر و اگر بیشتر از ۱ باشد متن را خلاقانه‌تر (و البته ریسکی‌تر) می‌کند.

```

1 def generate(model, params, idx, max_new_tokens, temperature=1.0, top_k=None, top_p=None):
2     for _ in range(max_new_tokens):
3         # Ensure context length does not exceed block_size
4         idx_cond = idx[:, -block_size:]
5
6         # Get raw predictions from the model
7         logits = model.apply(params, idx_cond)
8
9         # Adjust logits by temperature and focus on the last token
10        logits = logits[:, -1, :] / temperature
11
12        # Apply filtering if enabled
13        if top_k is not None: logits = top_k_filter(logits[0], top_k)
14        if top_p is not None: logits = top_p_filter(logits[0], top_p)
15
16        # Convert to probabilities and sample the next index
17        probs = jax.nn.softmax(logits)
18        next_token = jax.random.categorical(jax.random.PRNGKey(42), probs)
19
20        # Update the sequence
21        idx = jnp.concatenate([idx, next_token[None, None]], axis=1)
22    return idx
23

```

Listing 13: The Recursive Autoregressive Text Generation Function



شکل ۳۱: تأثیر دماهای مختلف بر توزیع احتمالات توکن بعدی

۳.۵ تحلیل نتایج تولید شده

بعد از پیاده‌سازی، مدل را با تنظیمات مختلف اجرا کردیم تا خروجی‌ها را بررسی کنیم. نمودارهای زیر نشان می‌دهند که چگونه انتخاب‌های مختلف برای پارامترهای فیلترینگ و دما بر روی کیفیت و تنوع متن نهایی تأثیر می‌گذارند.

```
=====
greedy-ish T=0.5
-----
ROMEO:
What thou dost thou art of the king of the duke.

KING RICHARD II:
And the prince and the king.

CAPULET:
I have men in the ground, here is person the country.

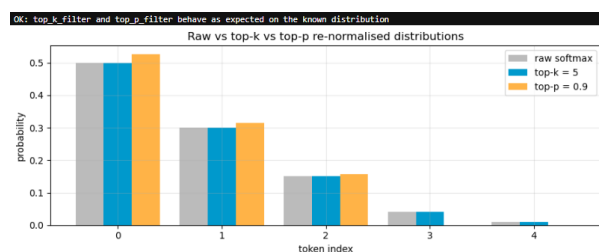
HENRY BOLINGBROKE:
I can to not me to
=====
top-k=20 T=0.8
-----
ROMEO:
Some second to the weep to say make me,
Suffer and to heaven the contreats in my more
Than out then against the foe a little and by the chounds
It is the father with sweet the enemy and of much:
And
=====
top-p=0.9 T=1.0
-----
ROMEO:
How made farewellower?

GLOUCESTER:
Now, march, good stays him behim:
Though hast speed thee out of shall be consquent.

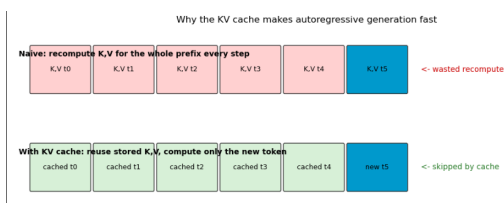
DUCHESS OF YORK:
Brother the qualines and watching or lips.

CAPULET:
Condure
=====
hot T=1.4
-----
ROMEO:
So thou Warwick, Warp; give me To pluck appt it.' auth,
When I have sleeping: or gloney, Yesiroh, Shojest;
Your brother, but and could was the's your
Gast's neck
Prophery, ho, moue; looks, withmors?
=====
```

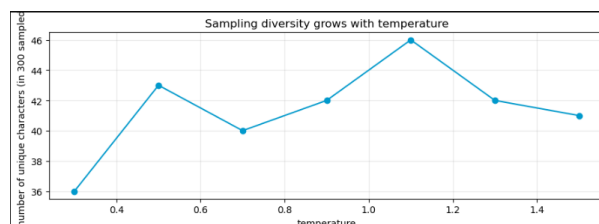
(ب) نمونه متن تولید شده با دمای متعادل (خلالقانه)



(آ) نمونه متن تولید شده با دمای پایین (محافظه‌کارانه)

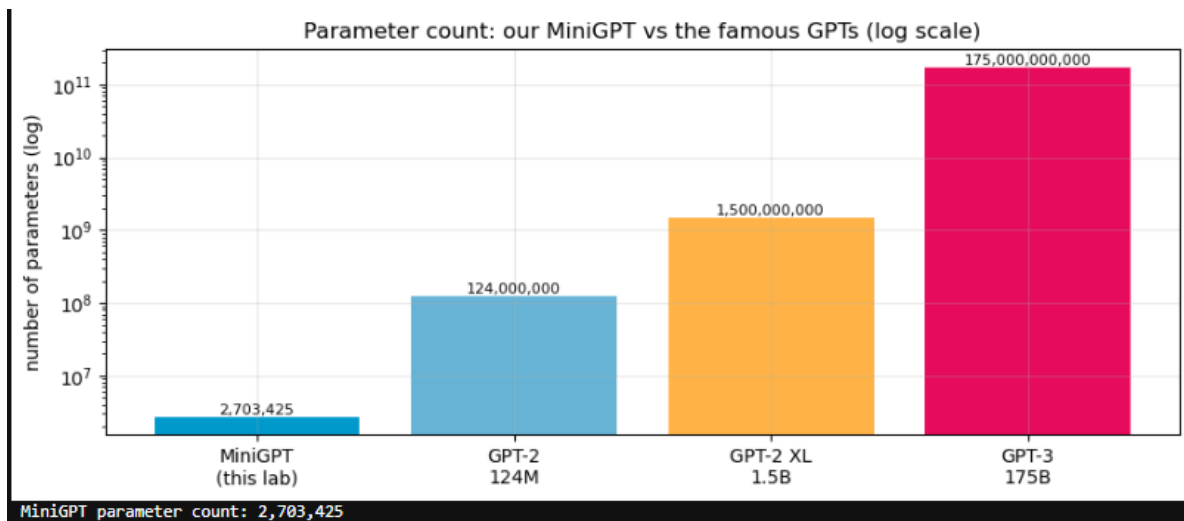


(ب) تغییرات آنتروپی متن در مراحل مختلف تولید



(آ) توزیع تنوع کلمات با استفاده از فیلتر Top-K

در نهایت، تصویر 'k۳۶' دید کلی از این پایپ‌لاین را نشان می‌دهد که چطور مدل آموزش دیده، ورودی را می‌گیرد و با پیمودن مسیر احتمالات، به یک متن معنادار می‌رسد.



شکل ۳۴: مقایسه مقیاس پارامتری MiniGPT ساخته شده با مدل‌های معروف خانواده GPT

۴.۵ جمع‌بندی کل پروژه

خاتمه و دستاورد نهایی

پروژه با موفقیت به سرانجام رسید. ما از پایه یاد گرفتیم که چگونه چطور متن را به اعداد تبدیل کنیم، چطور مکانیزم Attention را بسازیم، بلاک‌های ترانسفورمر را چطور مونتاژ کنیم و در نهایت چطور یک مدل مولد متن کاربردی داشته باشیم. این مسیر ۵ مرحله‌ای، دید عمیقی از معماری مدل‌های زبانی بزرگ (LLMs) به ما داد. مدل Mini-GPT ما حالا می‌تواند با الهام از سبک شکسپیر، متن‌های جدید و منحصر به فردی تولید کند که نشان‌دهنده درک مدل از الگوهای زبانی است. این پایان کار نیست، بلکه شروعی برای استفاده از این دانش در پروژه‌های پیشرفته‌تر هوش مصنوعی است.